

ENVELOPPE — Étape 3

Document remis par le surveillant, une fois le commit [JALON] v0.1 poussé. À ne pas distribuer avec le dossier de l'épreuve.

Le client a testé ta première version. Il revient avec deux choses.

1. Un bug

Il l'écrit comme un utilisateur, pas comme un développeur. À toi de traduire.

« J'ai ouvert une session ce matin avec deux étudiants côte à côte. Ils ont tapé le code presque en même temps et il n'y en a qu'un seul qui apparaît dans ma liste. J'ai réessayé une fois, cette fois les deux sont passés. Je ne comprends pas. »

Ce qu'on attend de toi, dans cet ordre :

1. Ouvrir une **issue** qui décrit le problème et **la manière de le reproduire** — avant de toucher au code.
2. Écrire un **test qui échoue** et qui démontre le bug.
3. Corriger, dans une branche dédiée, avec un commit qui référence l'issue.
4. Vérifier que le test passe.

Un correctif poussé sans issue et sans test compte pour la moitié des points. Le jury ne cherche pas si tu sais écrire la correction — l'IA la connaît — il cherche si tu sais **prouver** qu'un bug existait et qu'il n'existe plus.

2. Un changement de besoin

« Finalement, un seul relecteur ça ne marche pas : quand il ne rend rien, l'étudiant n'a aucune note. À partir de maintenant, **chaque exercice est relu par deux pairs différents**, et la note retenue est **la moyenne des deux**. Si un seul des deux a rendu, on affiche sa note en attendant, mais marquée comme provisoire. »

Ce changement casse la règle de gestion issue de Q6, et il touche **ta base, ton contrat d'API et ton frontend** en même temps.

Ce qu'on attend de toi :

1. **Mettre à jour ton analyse** — le cahier des charges (règles de gestion concernées, exigences, section 7) et les diagrammes qui deviennent faux, dans un commit qui dit clairement que c'est une conséquence de ce changement.
2. **Mettre à jour** `api/contrat.yaml` si la forme des réponses change.
3. **Écrire une nouvelle migration versionnée**. Tu ne modifies pas la migration existante, tu en ajoutes une. Si ta base est déjà remplie, elle doit survivre à la migration.
4. **Découper le travail en issues** et re-prioriser : ce changement est un **Must** qui arrive tard, donc quelque chose doit sortir du périmètre. **Écris ce que tu sacrifies et pourquoi** — dans ton

journal ou dans ton cahier des charges. Un périmètre réduit et assumé vaut mieux qu'un périmètre annoncé et non tenu.

5. **Séparer le correctif du changement** : deux branches, deux pull requests. Ce sont deux sujets différents, ils n'ont rien à faire dans le même commit.

Ce qui est regardé à cette étape (10 points)

L'issue existe avant le premier commit de correction	l'ordre dans l'historique le prouve
Le bug est reproduit par un test avant d'être corrigé	
La migration est ajoutée , jamais modifiée en place	
L'analyse est remise à jour, pas laissée périmée	
Le sacrifice de périmètre est écrit	
Correctif et évolution sont séparés	deux branches, deux PR

Tu n'es pas obligé de tout terminer. Tu es obligé de montrer comment tu t'y prends.